

INF781

SEGURIDAD DE SOFTWARE

GUÍA DE LABORATORIO

CAPTCHA

Defensa contra bots y abuso automatizado

Construcción del proyecto INF781-LaravelCaptcha desde cero

Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Carrera	Ingeniería Informática
Universidad	Universidad Autónoma Tomás Frías — Potosí, Bolivia
Stack tecnológico	Laravel 13 · PHP 8.3+ · PostgreSQL
Repositorio de referencia	github.com/HuascarFedor/INF781-LaravelCaptcha
Duración estimada	5 horas de laboratorio

1. Objetivos

1.1 Objetivo general

Diseñar e implementar mecanismos CAPTCHA en una aplicación Laravel 13, integrando una solución de tercero (Google reCAPTCHA v2) y una solución local (mews/captcha), aplicadas a los formularios sensibles de login, registro y contacto, evaluando su eficacia frente a ataques automatizados y comparando sus compromisos en términos de seguridad, accesibilidad, privacidad y experiencia de usuario.

1.2 Objetivos específicos

- Identificar las amenazas que justifican el uso de CAPTCHA: fuerza bruta, credential stuffing, registro masivo, spam, scraping y abuso de APIs.
- Configurar un proyecto Laravel 13 limpio con autenticación basada en Breeze y un formulario público de contacto.
- Integrar Google reCAPTCHA v2 (checkbox) en el formulario de registro y verificar la respuesta en el servidor mediante la API de siteverify.
- Integrar el paquete mews/captcha como alternativa local en el formulario de login, sin dependencia de servicios externos.
- Aplicar un CAPTCHA invisible o honeypot complementario al formulario de contacto y combinarlo con rate limiting.
- Implementar pruebas funcionales con PHPUnit que validen el rechazo de envíos sin CAPTCHA válido.
- Analizar críticamente las limitaciones de CAPTCHA: granjas humanas, IA visual, accesibilidad y privacidad.

2. Marco teórico

2.1 ¿Qué es un CAPTCHA?

CAPTCHA es el acrónimo de Completely Automated Public Turing test to tell Computers and Humans Apart. Se trata de una prueba de desafío-respuesta diseñada para diferenciar a un usuario humano de un agente automatizado (bot). Aunque el término data del año 2003 (Luis von Ahn et al., Carnegie Mellon), el concepto subyacente se remonta al test de Turing y a la idea de delegar al usuario un problema fácil para una persona pero difícil para una máquina.

En la práctica, un CAPTCHA no demuestra que el usuario sea humano: solo eleva el costo computacional o económico de cada intento automatizado hasta volver inviable un ataque a escala. Es, por tanto, un control de defensa en profundidad, nunca una barrera absoluta.

2.2 Taxonomía de CAPTCHA

Tipo	Mecanismo	Estado actual
Texto distorsionado	Transcribir letras y números con ruido visual	Obsoleto: OCR moderno lo rompe
Imágenes (selección)	Marcar imágenes que cumplen un criterio	Debilitado por modelos de visión por computadora
Audio	Transcribir audio con ruido (accesibilidad)	Vulnerable a speech-to-text

Tipo	Mecanismo	Estado actual
Matemático / lógico	Resolver una operación o pregunta de sentido común	Trivial para LLM y parsers
Checkbox (v2)	Marcar casilla + análisis de comportamiento previo	Vigente, pero no infalible
Invisible / score (v3)	Score de riesgo silencioso (0.0 a 1.0)	Vigente, requiere lógica del servidor
Slider / puzzle	Arrastrar pieza para completar imagen	Común en Asia, vulnerable a bots con trayectoria aprendida
Proof-of-Work	El cliente resuelve un problema costoso	Friendly Captcha, Turnstile (PoW + atestación)
Honeypot	Campo invisible que solo bots llenan	Complementario, no sustituye CAPTCHA real

2.3 Amenazas que mitiga

El CAPTCHA es un control transversal: actúa sobre la superficie de cualquier formulario o endpoint público. Las principales amenazas que limita son:

- Fuerza bruta y credential stuffing sobre login.
- Registro masivo de cuentas falsas (Sybil attacks).
- Spam en formularios de contacto, comentarios y foros.
- Scraping y extracción automatizada de contenido.
- Scalping y compra automatizada de inventario limitado.
- Enumeración de usuarios y correos para phishing dirigido.
- Denegación de servicio a nivel aplicación (Layer-7 DoS).
- Abuso de servicios de envío de correo o SMS para hostigamiento.

2.4 reCAPTCHA v2: arquitectura

Google reCAPTCHA v2 es el servicio gratuito más extendido. Funciona mediante un par de claves: una pública (site key) que se incrusta en la vista, y una privada (secret key) que vive en el servidor. El flujo es el siguiente:

1. El cliente carga el script api.js de reCAPTCHA y renderiza el widget en el formulario.
2. Al interactuar, Google evalúa señales de navegador (cookies, movimiento, reputación de IP) y emite un token g-recaptcha-response.
3. El formulario se envía al servidor Laravel junto con ese token.

4. El servidor llama a <https://www.google.com/recaptcha/api/siteverify> enviando el token y la secret key.
5. Google responde con un JSON: success (true/false), challenge_ts, hostname y, opcionalmente, error-codes.
6. Si success es true, el servidor procesa el formulario; de lo contrario, lo rechaza.

⚠ Importante: la verificación SIEMPRE es del lado servidor

Validar el CAPTCHA solo en el frontend es equivalente a no validar nada: un atacante puede omitir el JavaScript. La llamada a siteverify desde el backend es obligatoria.

2.5 CAPTCHA local con mews/captcha

Cuando un proyecto no puede o no quiere depender de servicios externos (privacidad, intranet, restricciones legales, normativas de soberanía de datos), un CAPTCHA local es razonable. El paquete mews/captcha genera imágenes con texto distorsionado del lado servidor, guarda el valor esperado en la sesión y proporciona reglas de validación de Laravel.

Ventajas: cero dependencia externa, control total, cero rastreo del usuario. Desventajas: vulnerable a OCR moderno, accesibilidad limitada y carga visual desagradable. Es una solución aceptable para escenarios de bajo riesgo o como segunda barrera detrás del rate limiting.

2.6 Honeypot como defensa complementaria

Un honeypot consiste en añadir al formulario un campo oculto por CSS que un humano nunca verá pero que los bots tienden a rellenar. Si ese campo llega con contenido al servidor, el envío se descarta silenciosamente. No reemplaza al CAPTCHA, pero filtra el tráfico más rudimentario sin fricción para el usuario legítimo.

3. Requisitos previos

- PHP 8.3 o superior con extensiones gd, mbstring, pdo_pgsql, openssl, curl.
- Composer 2.7+ y Node.js 20+ con npm.
- PostgreSQL 15+ disponible localmente.
- Cuenta de Google Cloud para registrar las claves de reCAPTCHA en <https://www.google.com/recaptcha/admin>
- Git y una cuenta de GitHub donde se publicará el repositorio INF781-LaravelCaptcha.
- Editor de código (VS Code, PHPStorm) y cliente HTTP (Postman o Bruno).

4. Construcción del proyecto

4.1 Creación del proyecto Laravel 13

Paso 1. Crear el proyecto base

Desde la terminal, en el directorio de trabajo del estudiante:

```
bash
composer create-project laravel/laravel:^13.0 INF781-LaravelCaptcha
cd INF781-LaravelCaptcha
git init && git add . && git commit -m "init: laravel 13 base"
```

Paso 2. Configurar PostgreSQL

Editar el archivo .env con las credenciales de la base de datos local:

```
.env
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=inf781_captcha
DB_USERNAME=postgres
DB_PASSWORD=postgres

APP_NAME="INF781 CAPTCHA Lab"
APP_URL=http://localhost:8000
SESSION_DRIVER=database
```

Paso 3. Instalar Breeze para autenticación

Breeze provee scaffolding limpio de login y registro con Blade, ideal para añadir CAPTCHA sin pelearse con SPA frameworks.

```
bash
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run build
php artisan migrate
```

Nota sobre Laravel 13

Laravel 13 usa el middleware PreventRequestForgery para CSRF, ya activo por defecto. La tabla sessions está incluida en la migración inicial, por lo que SESSION_DRIVER=database funciona sin migraciones extra.

5. Integración de reCAPTCHA v2 en el registro

5.1 Obtener las claves de Google

7. Acceder a <https://www.google.com/recaptcha/admin> con una cuenta Google.

8. Crear un sitio nuevo. Etiqueta sugerida: INF781-LaravelCaptcha.
9. Seleccionar el tipo reCAPTCHA v2 → "No soy un robot" (Casilla).
10. Agregar los dominios autorizados: localhost y 127.0.0.1 para desarrollo.
11. Aceptar los términos y copiar las dos claves: Site Key (pública) y Secret Key (privada).



La Secret Key es un secreto

Nunca debe aparecer en código del frontend, en commits públicos ni en capturas de pantalla. Solo en .env, fuera del control de versiones.

5.2 Configuración en Laravel

Paso 4. Añadir las claves al .env

```
.env
RECAPTCHA_SITE_KEY=6Lc...tu_site_key
RECAPTCHA_SECRET_KEY=6Lc...tu_secret_key
```

Paso 5. Crear el archivo de configuración config/services.php (sección)

Agregar al final del array de servicios en config/services.php:

```
config/services.php
'recaptcha' => [
    'site_key'    => env('RECAPTCHA_SITE_KEY'),
    'secret_key' => env('RECAPTCHA_SECRET_KEY'),
    'verify_url' => 'https://www.google.com/recaptcha/api/siteverify',
],
```

5.3 Regla de validación personalizada

Paso 6. Crear la regla Recaptcha

```
bash
php artisan make:rule Recaptcha
```

Implementar app/Rules/Recaptcha.php:

```
app/Rules/Recaptcha.php
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;
```

```
use Illuminate\Support\Facades\Http;

class Recaptcha implements ValidationRule
{
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        if (empty($value)) {
            $fail('Debes confirmar que no eres un robot.');
```

return;

```
        }

        $response = Http::asForm()->post(
            config('services.recaptcha.verify_url'),
            [
                'secret' => config('services.recaptcha.secret_key'),
                'response' => $value,
                'remoteip' => request()->ip(),
            ]
        );

        $data = $response->json();

        if (! ($data['success'] ?? false)) {
            $fail('La verificación CAPTCHA falló. Inténtalo de nuevo.');
```

}

```
    }
}
```

5.4 Modificar el controlador de registro

Editar `app/Http/Controllers/Auth/RegisteredUserController.php`. Añadir la regla al método `store()`:

```
app/Http/Controllers/Auth/RegisteredUserController.php
use App\Rules\Recaptcha;

// dentro de store():
$request->validate([
    'name' => ['required', 'string', 'max:255'],
    'email' => ['required', 'string', 'lowercase', 'email', 'max:255'],
    'unique:'.User::class,
    'password' => ['required', 'confirmed', Rules\Password::defaults()],
    'g-recaptcha-response' => ['required', new Recaptcha()],
]);
```

5.5 Modificar la vista de registro

Editar `resources/views/auth/register.blade.php`. Añadir antes del botón de envío:

```
resources/views/auth/register.blade.php
```

```
<div class="mt-4">
  <div class="g-recaptcha"
    data-sitekey="{{ config('services.recaptcha.site_key') }}">
  </div>
  @error('g-recaptcha-response')
    <p class="mt-1 text-sm text-red-600">{{ $message }}</p>
  @enderror
</div>

@push('scripts')
<script src="https://www.google.com/recaptcha/api.js" async defer></script>
@endpush
```

En `resources/views/layouts/guest.blade.php`, justo antes de `</body>`, añadir:

```
resources/views/layouts/guest.blade.php
@stack('scripts')
```

5.6 Probar el flujo

12. Ejecutar `php artisan serve` y visitar `http://localhost:8000/register`.
13. Verificar que el widget "No soy un robot" aparezca correctamente.
14. Intentar registrarse sin marcar el checkbox: el formulario debe rechazar con el mensaje de error.
15. Marcar el checkbox y completar el registro: debe pasar y crear el usuario.
16. Inspeccionar la red en DevTools: confirmar que se envía `g-recaptcha-response` y que el backend llama a `siteverify`.

6. CAPTCHA local con mews/captcha en login

6.1 Instalación del paquete

Paso 7. Instalar y publicar configuración

```
bash
composer require mews/captcha
php artisan vendor:publish --provider="Mews\Captcha\CaptchaServiceProvider"
```

Esto genera `config/captcha.php`. Para la guía utilizaremos el perfil `default`, pero existen también `flat`, `mini` e `inverse`. La configuración relevante:

```
config/captcha.php (extracto)
'default' => [
    'length' => 5,
```

```
'width'      => 120,  
'height'     => 36,  
'quality'    => 90,  
'math'       => false,  
'expire'     => 60,  
'encrypt'    => false,  
'sensitive'  => false,  
],
```

6.2 Ruta para servir la imagen

El paquete registra automáticamente la ruta GET `/captcha/{config?}`, que genera una imagen PNG fresca y guarda el valor esperado en sesión.

GET `/captcha/default` Devuelve una imagen PNG con el texto del desafío

GET `/captcha/api/default` Devuelve JSON con la imagen en base64 y la key

6.3 Modificar la vista de login

Editar `resources/views/auth/login.blade.php`. Añadir antes del botón submit:

```
resources/views/auth/login.blade.php  
  
<div class="mt-4">  
  <x-input-label for="captcha" :value="'Código de verificación'" />  
  <div class="flex items-center gap-3 mt-1">  
      
    <div class="border rounded" />  
    <button type="button" />  
      onclick="document.getElementById('captcha-img').src='{{  
captcha_src('default') }}'+Math.random()"  
      class="text-sm text-indigo-600 underline">  
        Recargar  
    </button>  
  </div>  
  <x-text-input id="captcha" />  
    class="block mt-2 w-full"  
    type="text"  
    name="captcha"  
    required  
    autocomplete="off" />  
  <x-input-error :messages="$errors->get('captcha')" class="mt-2" />  
</div>
```

6.4 Validación en el controlador

Editar `app/Http/Requests/Auth/LoginRequest.php`. Modificar el método `rules()`:

```
app/Http/Requests/Auth/LoginRequest.php
public function rules(): array
{
    return [
        'email' => ['required', 'string', 'email'],
        'password' => ['required', 'string'],
        'captcha' => ['required', 'captcha'],
    ];
}

public function messages(): array
{
    return [
        'captcha.required' => 'Debes ingresar el código de verificación.',
        'captcha.captcha' => 'El código de verificación es incorrecto.',
    ];
}
```

6.5 Probar el flujo

17. Visitar <http://localhost:8000/login>.
18. Confirmar que aparece la imagen y el botón de recarga.
19. Intentar autenticarse con credenciales correctas pero CAPTCHA inválido: debe fallar.
20. Intentar con captcha vacío: debe fallar con el mensaje personalizado.
21. Intentar con credenciales y captcha correctos: debe ingresar al dashboard.

Sensibilidad y accesibilidad

Por defecto `sensitive=false`: no diferencia mayúsculas y minúsculas. Mantener así para reducir frustración del usuario. Considerar añadir una alternativa audio en proyectos en producción para cumplir WCAG 2.1.

7. Formulario de contacto con honeypot + rate limiting

7.1 Diseño del formulario

Construiremos un formulario público de contacto (sin autenticación) que aplique tres capas de defensa: campo honeypot, rate limiting por IP y CAPTCHA invisible (reCAPTCHA v2 invisible) o local. Esto demuestra el principio de defensa en profundidad.

7.2 Migración del modelo Contact

```
bash
php artisan make:model Contact -m
```

```
database/migrations/...create_contacts_table.php
// database/migrations/xxxx_create_contacts_table.php
public function up(): void
{
    Schema::create('contacts', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->text('message');
        $table->ipAddress('ip')->nullable();
        $table->timestamps();
    });
}

bash
php artisan migrate
```

7.3 Controlador del formulario

```
bash
php artisan make:controller ContactController

app/Http/Controllers/ContactController.php
<?php

namespace App\Http\Controllers;

use App\Models>Contact;
use App\Rules\Recaptcha;
use Illuminate\Http\Request;

class ContactController extends Controller
{
    public function show()
    {
        return view('contact.show');
    }

    public function store(Request $request)
    {
        // 1) Honeypot: si el campo oculto llega lleno, es bot
        if (! empty($request->input('website'))) {
            return back()->with('status', 'Mensaje enviado.');
```

// Respuesta engañosamente exitosa, pero no guardamos nada

```
        }

        // 2) Validación + CAPTCHA local
        $data = $request->validate([
```

```

        'name'      => ['required', 'string', 'max:120'],
        'email'     => ['required', 'email', 'max:180'],
        'message'   => ['required', 'string', 'min:10', 'max:2000'],
        'captcha'   => ['required', 'captcha'],
    ]);

    Contact::create([
        'name'      => $data['name'],
        'email'     => $data['email'],
        'message'   => $data['message'],
        'ip'        => $request->ip(),
    ]);

    return back()->with('status', 'Tu mensaje fue enviado correctamente.');
```

7.4 Rutas con rate limiting

En routes/web.php:

```

routes/web.php
use App\Http\Controllers\ContactController;

Route::get('/contact', [ContactController::class, 'show'])->name('contact.show');

Route::post('/contact', [ContactController::class, 'store'])
    ->middleware('throttle:5,1') // 5 intentos por minuto y por IP
    ->name('contact.store');
```

7.5 Vista con honeypot oculto por CSS

Crear resources/views/contact/show.blade.php:

```

resources/views/contact/show.blade.php
<x-guest-layout>
    @if (session('status'))
        <div class="mb-4 text-sm text-green-700">{{ session('status') }}</div>
    @endif

    <form method="POST" action="{{ route('contact.store') }}">
        @csrf

        {{-- HONEYPOT: invisible para humanos, irresistible para bots --}}
        <div style="position:absolute; left:-9999px; top:-9999px;" aria-
hidden="true">
            <label for="website">Sitio web</label>
            <input type="text" name="website" id="website" tabindex="-1"
autocomplete="off">
        </div>
    </form>
</x-guest-layout>
```

```

</div>

<div>
  <x-input-label for="name" value="Nombre" />
  <x-text-input id="name" name="name" type="text" required />
  <x-input-error :messages="$errors->get('name')" class="mt-2" />
</div>

<div class="mt-4">
  <x-input-label for="email" value="Correo" />
  <x-text-input id="email" name="email" type="email" required />
  <x-input-error :messages="$errors->get('email')" class="mt-2" />
</div>

<div class="mt-4">
  <x-input-label for="message" value="Mensaje" />
  <textarea id="message" name="message" rows="5"
    class="block w-full border-gray-300 rounded-md shadow-
sm"></textarea>
  <x-input-error :messages="$errors->get('message')" class="mt-2" />
</div>

<div class="mt-4">
  <x-input-label for="captcha" value="Código de verificación" />
  
  <x-text-input id="captcha" name="captcha" type="text" required />
  <x-input-error :messages="$errors->get('captcha')" class="mt-2" />
</div>

  <x-primary-button class="mt-4">Enviar</x-primary-button>
</form>
</x-guest-layout>

```

¿Por qué responder "éxito" al bot del honeypot?

Si un atacante automatizado recibe un mensaje de error explícito al caer en el honeypot, ajustará su bot para evitarlo. En cambio, si recibe una respuesta aparentemente exitosa, no sabrá que su intento fue descartado y seguirá gastando recursos en una ruta inerte. Es defensa por engaño.

8. Pruebas automatizadas con PHPUnit

8.1 Crear el test feature

```

bash
php artisan make:test CaptchaProtectionTest

```

Editar tests/Feature/CaptchaProtectionTest.php:

```
tests/Feature/CaptchaProtectionTest.php
<?php

namespace Tests\Feature;

use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class CaptchaProtectionTest extends TestCase
{
    use RefreshDatabase;

    public function test_register_falla_sin_recaptcha_token(): void
    {
        $response = $this->post('/register', [
            'name'          => 'Test User',
            'email'         => 'test@example.com',
            'password'      => 'password123',
            'password_confirmation' => 'password123',
            // sin g-recaptcha-response
        ]);

        $response->assertSessionHasErrors('g-recaptcha-response');
        $this->assertDatabaseMissing('users', ['email' => 'test@example.com']);
    }

    public function test_login_falla_sin_captcha(): void
    {
        $response = $this->post('/login', [
            'email' => 'usuario@example.com',
            'password' => 'cualquier',
            // sin captcha
        ]);

        $response->assertSessionHasErrors('captcha');
    }

    public function test_contacto_honeypot_descarta_envio_silenciosamente(): void
    {
        $response = $this->post('/contact', [
            'name'      => 'Bot',
            'email'     => 'bot@spam.com',
            'message'   => 'mensaje masivo',
            'website'   => 'http://spam.com', // honeypot lleno
        ]);

        $response->assertSessionHas('status');
        $this->assertDatabaseCount('contacts', 0);
    }
}
```

Paso 8. Ejecutar la suite

```
bash
php artisan test --filter=CaptchaProtectionTest
```

Los tres tests deben pasar en verde. Captura la salida del terminal y adjúntala al informe.

9. Análisis crítico y discusión

Tras completar la implementación, el estudiante debe responder por escrito (mínimo 250 palabras por pregunta) las siguientes cuestiones en el informe:

22. ¿Qué amenazas específicas mitiga cada uno de los tres formularios protegidos y cuál es la amenaza residual que el CAPTCHA no resuelve?
23. Compara reCAPTCHA v2 (Google) frente a mews/captcha en cinco dimensiones: seguridad, accesibilidad, privacidad del usuario, dependencia externa y experiencia. ¿En qué escenario preferirías cada uno?
24. Describe al menos dos formas conocidas de eludir un CAPTCHA de imágenes y propone defensas adicionales (rate limiting, MFA, análisis de comportamiento) que las contrarresten.
25. ¿Qué problemas de privacidad introduce reCAPTCHA al usuario final? Investiga sobre el rastreo de Google y las críticas del GDPR en la Unión Europea.

10. Entregables

Entregable	Formato	Ubicación
Repositorio público en GitHub	Git	github.com/<usuario>/INF781-LaravelCaptcha

Estructura mínima del README.md

- Título y descripción del proyecto.
- Requisitos y comandos de instalación.
- Instrucciones para obtener y configurar las claves de reCAPTCHA.
- Comandos para ejecutar la app y los tests.
- Capturas de pantalla de los tres formularios protegidos.
- Sección de Análisis crítico (preguntas del punto 9).
- Licencia y autor.

11. Rúbrica de evaluación

La calificación se distribuye sobre 100 puntos. Para aprobar la guía se requiere un mínimo de 60.

Criterio	Indicador	Pts
Configuración del proyecto	Laravel 13 funcional, PostgreSQL, Breeze, repo inicializado	10
Integración reCAPTCHA v2	Widget renderizado y verificación server-side completa	18
Integración mews/captcha	Imagen local en login, validación con regla 'captcha'	15
Formulario de contacto	Honeypot + rate limiting + captcha funcionando	15
Pruebas PHPUnit	Tres tests pasando, cubren los tres formularios	12
Análisis crítico	Cuatro preguntas respondidas con profundidad y citas	12
Calidad de código	Convenciones PSR-12, separación de responsabilidades, sin secretos en repo	8
Documentación y README	README reproducible, capturas, estructura clara	5
Sustentación oral	Demostración fluida y respuestas correctas a preguntas	5
TOTAL		100

12. Referencias y lecturas recomendadas

- von Ahn, L., Blum, M., Hopper, N., Langford, J. (2003). CAPTCHA: Using Hard AI Problems for Security. EUROCRYPT 2003.
- Google. Documentación oficial de reCAPTCHA.
<https://developers.google.com/recaptcha>
- Mews. Repositorio mews/captcha. <https://github.com/mewebstudio/captcha>
- Laravel 13. Documentación oficial — Validation Rules y Rate Limiting.
<https://laravel.com/docs/13.x>

- OWASP. Automated Threats to Web Applications (OAT) Project. <https://owasp.org/www-project-automated-threats-to-web-applications/>
- Cloudflare. Turnstile y la economía del bot. <https://blog.cloudflare.com/turnstile-private-captcha-alternative/>
- Bock, K. et al. (2017). uncaptcha: A Two-Year Retrospective on Audio CAPTCHA. USENIX Security.